

自注意力和位置编码

自注意力用同一组输入生成查询、键和值，让序列中的每个词元直接关注所有词元。它可以并行计算，也缩短了远距离信息传递的路径，但计算量随序列长度呈二次增长，处理很长的序列时会很慢。要让模型利用序列的顺序信息，可以在输入表示中加入位置编码。

自注意力如何表示序列

卷积神经网络（CNN）和循环神经网络（RNN）常用于序列编码。注意力机制提供了另一种方式：把词元序列送入注意力汇聚，让同一组词元同时充当查询、键和值。每个查询关注所有键值对，生成一个输出。由于三者来自同一组输入，这种机制称为**自注意力**（self-attention；Lin、Feng、Santos 等，2017；Vaswani、Shazeer、Parmar 等，2017），也称**内部注意力**（intra-attention；Cheng、Dong、Lapata，2016；Parikh、Tackstrom、Das 等，2016；Paulus、Xiong、Socher，2017）。

设输入序列为 $\mathbf{x}_1, \dots, \mathbf{x}_n$ ，其中 $\mathbf{x}_i \in \mathbb{R}^d$ ， $1 \leq i \leq n$ 。自注意力输出同样长度的序列 $\mathbf{y}_1, \dots, \mathbf{y}_n$ ：

$$\mathbf{y}_i = f(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \dots, (\mathbf{x}_n, \mathbf{x}_n)) \in \mathbb{R}^d.$$

这里， f 是注意力汇聚函数， $\mathbf{x}_i$ 是当前查询，各个 $(\mathbf{x}_j, \mathbf{x}_j)$ 分别构成键值对。

实现时，可以把同一个张量分别作为多头注意力的查询、键和值。输入张量的形状为“批量大小 $\times$ 序列长度 $\times$ 隐藏维度”，输出形状与之相同。例如，使用隐藏维度为 100、头数为 5、暂退率为 0.5 的多头注意力，将形状为 (2, 4, 100) 的全一张量作为三组输入，并指定两个样本的有效长度分别为 3 和 2，输出形状仍为 (2, 4, 100)。

CNN、RNN 与自注意力的比较

比较三种架构时，统一考虑以下任务：将包含 n 个词元的序列映射为等长序列，每个输入和输出词元都用 d 维向量表示。

除了计算复杂度，还需要考察顺序操作的数量和最大路径长度。顺序操作越多，并行计算受到的限制越大；序列中任意两个位置之间的路径越短，远距离依赖关系就越容易学习（Hochreiter、Bengio、Frasconi 等，2001）。

架构	计算复杂度	顺序操作数量	最大路径长度
CNN，卷积核大小为 k	$\mathcal{O}(knd^2)$	$\mathcal{O}(1)$	$\mathcal{O}(n/k)$
RNN	$\mathcal{O}(nd^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
自注意力	$\mathcal{O}(n^2d)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

卷积神经网络。对卷积核大小为 k 、输入和输出通道数均为 d 的卷积层，长度为 n 的序列带来的计算复杂度为 $\mathcal{O}(knd^2)$ 。同一层中不同位置的计算可以并行，顺序操作数量为 $\mathcal{O}(1)$ 。卷积通过层与层之间的连接传递信息，最大路径长度为 $\mathcal{O}(n/k)$ 。例如，在卷积核大小为 3 的双层卷积网络中， $\mathbf{x}_1$ 和 $\mathbf{x}_5$ 可以处于同一感受野内；这里忽略填充词元。

循环神经网络。更新一个隐状态，需要将 $d \times d$ 的权重矩阵与 d 维隐状态相乘，计算复杂度为 $\mathcal{O}(d^2)$ 。处理全部 n 个词元，单层的计算复杂度为 $\mathcal{O}(nd^2)$ 。隐状态必须依次更新，因此有 $\mathcal{O}(n)$ 个无法并行化的顺序操作，最大路径长度也为 $\mathcal{O}(n)$ 。

自注意力。查询、键和值都是 $n \times d$ 矩阵。在缩放点积注意力中，先将 $n \times d$ 的查询矩阵与 $d \times n$ 的键转置矩阵相乘，得到 $n \times n$ 的矩阵；再用注意力权重矩阵乘以 $n \times d$ 的值矩阵。因此，计算复杂度为 $\mathcal{O}(n^2d)$ 。每个词元都直接连接其他词元，不同位置的计算可以并行，顺序操作数量和最大路径长度均为 $\mathcal{O}(1)$ 。

CNN 和自注意力都具有并行计算的优势，自注意力的最大路径长度最短。不过，自注意力的计算量随序列长度呈二次增长，所以在很长的序列上计算会非常慢。

用位置编码补充顺序信息

RNN 按顺序逐个处理词元，自注意力则不依赖这种顺序操作。为了利用序列中的顺序，可以在输入表示中加入**位置编码**（positional encoding），注入绝对或相对位置信息。位置编码既可以通过学习获得，也可以固定下来。下面介绍 Vaswani、Shazeer、Parmar 等（2017）提出的正弦、余弦固定位置编码。

设 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 包含 n 个词元的 d 维嵌入表示。构造形状相同的位置嵌入矩阵 $\mathbf{P} \in \mathbb{R}^{n \times d}$ ，将输入改为 $\mathbf{X} + \mathbf{P}$ 。矩阵 $\mathbf{P}$ 的行表示词元在序列中的位置，列表示位置编码的不同维度。按从 0 开始的索引，第 i 行中相邻两列的元素为：

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right),$$

$$p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right).$$

实现时，可以预先计算足够长的位置编码矩阵。例如，位置编码层默认预留 1000 个位置，在偶数索引列填入正弦值，在奇数索引列填入余弦值。前向计算只截取与当前序列等长的部分，将其加到输入上，再应用暂退层；在 PyTorch 等框架中，还需让位置编码与输入位于同一设备。

以编码维度 $d = 32$ 、序列长度为 60、暂退率为 0 的例子观察这些曲线：索引为 6、7 的两列比索引为 8、9 的两列变化得更快。每对相邻列分别使用正弦和余弦，因此同一对曲线之间存在相位偏移。

绝对位置信息

不同维度使用不同频率，可以借助二进制表示来理解。整数 0 到 7 的三位二进制表示依次为 000、001、010、011、100、101、110、111。最低位每经过一个数切换一次，第二低位每经过两个数切换一次，第三低位每经过四个数切换一次。比特位越高，切换频率越低。

正弦、余弦位置编码具有类似的规律：随着编码维度增加，三角函数的频率逐渐降低。它用连续的浮点数表示位置，这类连续表示比二进制表示法更节省空间。

相对位置信息

这种位置编码除了表示绝对位置，还允许模型学习相对位置信息。原因是：对任意固定偏移量 δ ，位置 $i + \delta$ 的编码都可以由位置 i 的编码经过线性变换得到，而且该变换不依赖起始位置 i 。

令

$$\omega_j = \frac{1}{10000^{2j/d}}.$$

对于每一对编码 $(p_{i,2j}, p_{i,2j+1})$ ，有：

$$\begin{aligned} & \begin{bmatrix} \cos(\delta\omega_j) & \sin(\delta\omega_j) \\ -\sin(\delta\omega_j) & \cos(\delta\omega_j) \end{bmatrix} \begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix} \\ &= \begin{bmatrix} \cos(\delta\omega_j)\sin(i\omega_j) + \sin(\delta\omega_j)\cos(i\omega_j) \\ -\sin(\delta\omega_j)\sin(i\omega_j) + \cos(\delta\omega_j)\cos(i\omega_j) \end{bmatrix} \\ &= \begin{bmatrix} \sin((i+\delta)\omega_j) \\ \cos((i+\delta)\omega_j) \end{bmatrix} = \begin{bmatrix} p_{i+\delta,2j} \\ p_{i+\delta,2j+1} \end{bmatrix}. \end{aligned}$$

这个 2×2 矩阵只取决于偏移量 δ 和该对编码的频率 ω_j ，不依赖位置索引 i 。因此，同一固定偏移可以用同一线性变换表示。

小结

- 自注意力的查询、键和值来自同一组输入，每个词元都能直接关注序列中的其他词元。
- CNN 和自注意力都便于并行计算，自注意力的最大路径长度最短；但自注意力的计算量随序列长度呈二次增长，处理很长的序列时会很慢。
- 在输入表示中加入位置编码，可以补充绝对或相对位置信息。正弦、余弦位置编码使用随维度降低的频率表示位置，并能通过与起始位置无关的线性变换表示固定的位置偏移。

练习

1. 假设通过堆叠带有位置编码的自注意力层，构建一个表示序列的深度架构，可能会出现什么问题？
2. 设计一种可学习的位置编码方法。